# Object-Centric Grounding for End-to-End Driving

## Master's thesis progress and research stay proposal

Richard Cernansky

Faculty of Informatics and Information Technologies
Slovak University of Technology in Bratislava

Interview, Mobile Perception Systems Lab, TU Eindhoven

26 October 2026

# Agenda

1. **Master's thesis**
   - Research problem and objectives
   - Technical approach and methodology
   - Progress, preliminary results and how I read them
2. **Research stay proposal**
   - Motivation and the research gap
   - Hypotheses, methodology, expected contributions
3. **Connection between the two**
   - What the thesis delivers for the proposal
   - Scientific questions for the stay, fit with the lab

# Outline

1 Master's Thesis

2 Research Stay Proposal

3 Connection

# Trajectory prediction

Modular stacks **detect**, then **track**, then **predict** [1, 2].

- The interface between stages is just a list of boxes and identities
- Whatever information NOT encoded in a box is discarded
- *Errors propagate forward* and cannot be corrected downstream

**End-to-end perception and prediction** removes that interface.
**ViP3D**[3] carries persistent agent queries through time, and the trajectory decoder reads the query features.

**Problem with ViP3D's Agent Queries**

- learned embedding of fixed size
- identical in every frame, scene
- reference points come from that embedding (so Query X may start in the same place on an empty motorway and at a busy intersection)

# Objectives

**Goal of the thesis**

A deep learning architecture for end-to-end perception and prediction in which the agent queries originate in the point cloud rather than in a learned embedding.

**3 research questions:**

1. What do the query-based representation **derived from LiDAR** achieve for the model?
2. Once the queries already carry geometric evidence, what **does the camera contribute**?
3. A query that starts near its object **needs less layers to correct**. How does it effect decoder depth and in inference latency?

# Query initialisation is interesting

Input-dependent reference points are **known to be better** and to reduce the decoder depth needed for convergence.

**Our reading of the detection literature:**

- **DETR**[4] queries are learned vectors with no explicit position at all.
- **Deformable DETR**[5] makes the position explicit as a reference point, predicted from the query embedding - **input-independent**
- Its two-stage variant takes reference points from the centres of the top-scoring encoder proposals instead, and outperforms the single-stage one.
- **TransFusion**[6] do the initialisation in BEV, heatmap peaks into object queries.

Nobody has applied query initialisation to end-to-end perception and prediction.

# So how is the query generated?

A class-wise heatmap is predicted over the BEV grid with CenterPoint-style[7] Gaussian targets. For each of the top-$K$ peaks:

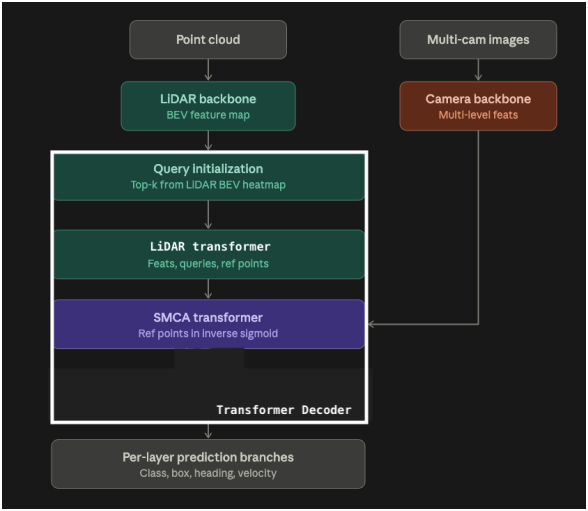
- the reference point is the metric position of the peak,
- the content is the BEV feature sampled at that cell plus an embedding of its class.

**What makes this work with tracking:**

- Only the slots left free by the tracker are filled
- Agents already being followed keep the queries they carry (suppression)
- initialisation concerns newly appearing objects

Local-max NMS is 3×3 and excludes pedestrian and bicycle classes

# InitVip3D Architecture




Queries are born from the LiDAR BEV heatmap ▸ see backup

Two decoder layers refine the points, deformable BEV attention and then camera SMCA.

## Loss:

$$\mathcal{L} = \mathcal{L}_{\text{hm}} + \underbrace{(\mathcal{L}_{\text{cls}} + \mathcal{L}_{\text{box}})}_{\text{detection}} + \underbrace{(\mathcal{L}_{\text{reg}} + \mathcal{L}_{\text{prob}})}_{\text{prediction}}$$

- $\mathcal{L}_{\text{det}}$ and $\mathcal{L}_{\text{pred}}$ only for matched agents
- Trajectory prediction $\mathcal{L}_{\text{pred_regression}}$ is SmoothL1
- $\mathcal{L}_{\text{hm}}$ is focal loss

# Methodology

**Training**, two stages following the TransFusion recipe[6] **but use batch size 8 on 8x Nvidia H200**

- **Stage 1: LiDAR model** alone, 20 epochs
- **Stage 2**: stage 1 frozen, camera layer on top, 6 epochs (so fusion cannot destabilise)
- No data augmentation in any run (future work)

**Data**, nuScenes[8]

- 700 training and 150 validation scenes, keyframes at 2 Hz
- Persistent instance identities, **3-frame clips**, **6 s futures**
- Seven classes: car, truck, bus, trailer, motorcycle, bicycle, pedestrian

**Metrics**

- **EPA**[3] is the primary criterion see backup
- minADE, minFDE, MR cover matched agents only
- Detection and tracking metrics alongside

**Evaluation Protocol**

- Identical code, thresholds, class set
- Best epoch by EPA
- **Latency**: one fixed frame replayed, over 200 timed iterations after warm-up, protocol the published rows use

# Preliminary results

Table: nuScenes validation, seven classes, no augmentations, 6 s horizon, $K = 6$ modes. The top block is published numbers, the lower two are my stage-1 LiDAR-only and stage-2 fusion runs, every one of at effective batch 8 under identical evaluation code.

| Run | Queries | EPA ↑ | minADE ↓ | minFDE ↓ | MR ↓ | Params (M) | **Latency (ms) ↓** |
|---|---|---|---|---|---|---|---|
| ViP3D *as published*[3] (baseline) | 300 | 0.245 | 1.47 | 2.21 | 0.237 | 41.77 | 64.3 |
| Li-ViP3D++ *as published*[9] | 300 | 0.505 | 1.11 | 1.80 | 0.209 | — | 76.28 |
| Stage 1, LiDAR only | 100 | 0.256 | 1.022 | 1.937 | 0.239 | 40.80 | 81.1 |
| **Stage 1, LiDAR only, 5 sweeps** | 300 | 0.263 | 1.001 | 1.947 | 0.204 | 40.80 | 58.2 |
| **Stage 1, LiDAR only** | 300 | **0.293** | 0.956 | 1.776 | 0.183 | 40.80 | 90.0 |
| **Stage 2, + SMCA camera** | 300 | **0.318** | **0.882** | **1.590** | **0.176** | 41.66 | 130.3 |

# Preliminary results: detection

Table: Per-class detection AP, same conditions as above. stage-1 LiDAR-only at effective batch 8, epoch 14. The three published columns are Table III of Li-ViP3D++[9] under its original-input setting.

| AP ↑ | ViP3D[3] | Li-ViP3D[10] | Li-ViP3D++[9] | **Stage 1 LiDAR only** |
|---|---|---|---|---|
| car | 0.590 | 0.590 | **0.717** | 0.433 |
| pedestrian | 0.311 | 0.305 | **0.433** | 0.381 |
| truck | 0.324 | 0.342 | **0.515** | 0.095 |
| bus | 0.342 | 0.335 | **0.619** | 0.000 |
| motorcycle | 0.268 | 0.278 | **0.355** | 0.000 |
| trailer | 0.104 | 0.114 | **0.227** | 0.000 |
| bicycle | 0.296 | 0.315 | **0.325** | 0.000 |
| mAP *as each source reports it* | 0.472 | 0.472 | 0.616 | 0.130 |

# Reading the results

1. **Agents of the most represented classes get most of the gradient.** Need for augmentation for less represented classes (future step).
2. **Query init. matters more than the fusion does.**
3. **Camera fusion brings small benefit with increased latency.** Camera fusion brings +40.3ms latency for only +0.025 EPA
4. **Reduction to 100 queries** $\rightarrow$ $-9$ **ms latency**, but EPA drops 0.293 to 0.256 (savings cost)
5. **Reduction in number of sweeps still beats ViP3D in every metric.**
6. **PointPillar LiDAR backbone is weak on detections.** Next step is to try Voxel Net (higher cost)

# Outline

1 Master's Thesis

2 Research Stay Proposal

3 Connection

# Motivation

Vision-language-action models are a route to interpretable end-to-end driving. **Alpamayo-R1**[11] generates a **Chain-of-Causation reasoning trace**, and **flow-matching action expert** decodes a trajectory conditioned on it.

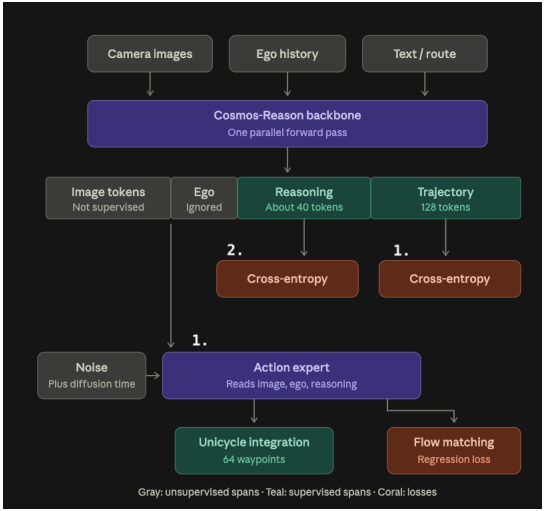
The reported gains concentrate exactly where imitation learning is weakest:

- 12 % improvement in planning accuracy on challenging cases
- 35 % reduction in close-encounter rate in closed-loop simulation.

**Important structural observation**

Trace is a functional intermediate representation between perception and planning. **AR1 restores the hard interface**, but expresses it in natural language.

# Alpamayo-R1 architecture




One backbone pass emits the reasoning and the trajectory tokens.

The grey spans carry no supervision.

The action expert denoises acceleration - decodes a flow field, Euler-integrated into controls, then unicycle-integrated into waypoints.

1. **Action Modality Injection:** Cross-entropy loss for discrete trajectory tokens and conditional flow matching for the continuous action expert
2. **Eliciting Reasoning:** Supervised Fine-Tuning (SFT) utilizing Cross-entropy loss on the structured Chain of Causation (CoC) dataset
3. **RL-based Post-Training:** Group Relative Policy Optimization (GRPO) using verifiable rewards for reasoning quality, action consistency, and safety

# The research gap

A trace says *"the black SUV from the left is going straight"*[11]. Nothing in the architecture requires that SUV to exist.

- Reasoning conditions on the image tokens that are shaped **only by the language-modelling objective.**
- **Weak visual grounding** is listed among the limitations of supervised fine-tuning, and the mitigation is post-hoc, a reasoning model grading traces during RL (nothing constrains the reasoning at inference time)
- **No temporal preservation** - causal locality in the labelling protocol and a KV-cache rebuilt each frame.

**Observation**

Driving VLAs couple perception to reasoning through an unverifiable, memoryless interface, grounded in neither sensor evidence nor time.

# Objectives and hypotheses

**Objectives:**

- Introduce an explicit, sensor-grounded, temporally propagated object representation into the VLA sequence
- evaluate whether it improves the quality of both the reasoning and the resulting trajectories

**H1** **Grounding failure is measurable.** Traces from the released AR1 model refer to object categories absent from the scene at a rate significantly above that of the human-verified

**H2** **An object-centric span reduces it.** Conditioning the reasoning on a span of detection-derived agent queries lowers that rate without degrading performance

**H3** **Temporal propagation extends the benefit beyond the observation window.** Propagating queries across frames helps re-detection

# Phase 1: measure the failure

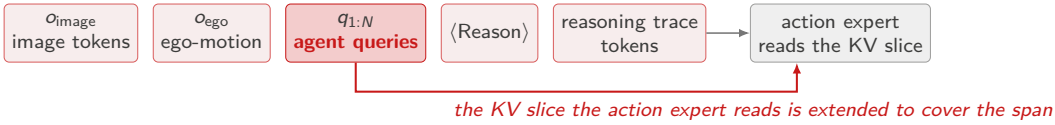
**No training required.** Run the released checkpoint over clips that also carry 3D annotations.

1. Parse each **model generated trace** for referenced categories.
2. Score a reference as unsupported when no GT annotated object of that category exists in that region at that time.
3. Apply the identical measurement to the **human-written traces**, compute the gaps between model and human traces unsuported ratios and report

**Note**

It is cheap and needs no new annotation.

# Phase 2: the object span




Agent queries are projected into the backbone embedding space and inserted before the reasoning tokens. Causal attention then exposes every query to every reasoning token, so grounding reaches the language path and the control path at once.

**Auxiliary detection loss**

Their future work[11] suggests depth or scene-flow auxiliaries to improve the visual backbone.

# Controls, because the span might simply be ignored

**Span masking** Replace the queries with null embeddings at inference and measure the change in the generated traces. No change means the span is inert.

**Span corruption** Substitute queries taken from a different scene. A trace that adopts the foreign objects shows the span is genuinely read.

# Phase 3: temporal propagation, and the grounding reward

*depends on the time remaining*

**Temporal propagation**

- Queries are propagated between frames with ego-motion compensation
- Each query keeps a FIFO memory bank of its own past states, attended over by temporal cross-attention [3]

**Grounding reward**

- Parse model generated trace for object references
- Compare against the query set, score the match
- the RL stage gains a grounding reward

**Example**

Trace: *"yielding to the pedestrian on the right, lead vehicle stopped"*
Referenced: {pedestrian, vehicle} Query set: {car, truck}
Score: pedestrian unmatched, penalty applied.

# Evaluation protocol and expected contributions

**Evaluation.**

- *Grounding*: unsupported-reference rate per category and scenario
- *Open-loop*: $\text{minADE}_6$ and minFDE over 6.4 s,
- *Closed-loop*: close-encounter rate, off-road rate and AlpaSim score, with an occlusion breakdown.
- *Cost*: added tokens, added parameters, end-to-end latency against the reported 99 ms budget.

**Expected contributions.**

1. A **quantitative characterisation** of object-level hallucination in a released driving VLA, with a reusable protocol. This stands regardless of H2 and H3.
2. An **architecture and training recipe** for a sensor-grounded, temporally propagated object span, with ablations
3. An investigation of the token $\leftrightarrow$ latency **budget trade-off**
4. A workshop or conference **paper**

# Outline

1 Master's Thesis

2 Research Stay Proposal

3 Connection

# How the stay builds on the thesis

The thesis produces a representation AR1 lacks.

## My thesis delivers

- A set of agent queries grounded in sensor data by a detection loss
- Carrying persistent identity across frames through a memory bank

## The research on stay requires

- A query span to insert into a token sequence
- A supervision signal that grounds it in the scene
- State that survives beyond the observation window
- Queries that reflect a particular scene

**In one sentence**

The thesis built the representation, which could be used to extend AR1 with the grounded, persistent object set that its reasoning currently has to invent.

# Summary: Scientific questions to be addressed during the stay

1. How often does a released driving VLA **refer to objects** that are **not there**?
2. Does conditioning the reasoning on an explicit, **detection-supervised object span** reduce the hallucinations, and can the span be shown to be used?
3. Does **propagating the queries across frames** extend the benefit to occlusion and reappearance?
4. What does the **span cost in tokens**, parameters and latency, and is that affordable on automotive embedded hardware?

# Fit with the Mobile Perception Systems Lab

- The lab's long-term goal is spatio-temporal reasoning that lets a mobile platform anticipate future events, identified as a bottleneck in crowded inner-city settings.
- group's current work on driving models spans **perception, simulation and embedded deployment** (prototype vehicle and digital twin)
- this proposal sits in the perception and reasoning parts
- The test vehicle carries a 360-degree camera setup with LiDAR and radar. My preliminary work is LiDAR-based and transfers directly.
- Closed-loop evaluation could run in the TU/e digital twin alongside the public AlpaSim scenario sets.

# Thank you

Technical discussion and questions

# Backup: why EPA is the primary metric

EPA as defined by ViP3D[3]:

$$\mathrm{EPA} = \max\left(0,\ \frac{N_{\mathrm{hit}} - \alpha N_{FP}}{N_{\mathrm{GT}}}\right), \qquad \alpha = 0.5$$

- Predicted agents are matched to annotated agents by Hungarian assignment on centre distance, gated at 2 m, in the current frame.
- $N_{\mathrm{hit}}$ counts an agent as a hit only if its $\mathrm{minFDE}_K$ stays within 2 m at the horizon.
- minADE, minFDE and MR average only over matched agents, so a model that detects fewer objects can improve all three.
- $N_{\mathrm{GT}}$ counts the annotated agents that survive the whole horizon, and false positives are subtracted, so EPA penalises both failure modes at once.

# Backup: the tracking interface

The model's inference behaviour is controlled by hard parameters

| Parameter | Value |
|---|---|
| confirm a new track | 0.40 |
| keep a track alive | 0.35 |
| miss tolerance | 5 frames |
| heatmap NMS kernel | $3\times3$ |
| active-track suppression | $5\times5$ cells |

# Backup: batch size and query count sweep

Stage 1, LiDAR only, nuScenes validation. Learning rate was sqrt-scaled with the batch size.

| Stage-1 variant | minADE ↓ | EPA ↑ | Note |
|---|---|---|---|
| 300 queries, effective batch 8 | 0.956 | 0.293 | best, the row in the results table |
| 300 queries, batch 16 | 0.997 | 0.277 | |
| 300 queries, batch 32 | 1.114 | 0.236 | |
| 300 queries, batch 64 | 1.061 | 0.256 | |
| 100 queries, batch 8 | 1.022 | 0.256 | |
| 100 queries, batch 16 | 1.528 | 0.234 | |

Accuracy degrades close to monotonically with batch size even with the learning rate scaled, which is consistent with the usual small-batch-noise argument.

# Backup: configuration details

**LiDAR branch**

- Point cloud range $\pm 51.2$ m, pillars of 0.2 m, 20 points per pillar.
- PillarFeatureNet $5 \to 64$, PointPillarsScatter to $512 \times 512$.
- SECOND with strides $[2, 2]$, SECONDFPN to $384 \times 256 \times 256$, so 0.4 m per BEV cell.
- 10/5 accumulated sweeps, 5 features per point.

**Heatmap head**

- CenterPoint nuScenes task grouping, so pretrained per-task weights still line up.
- Gaussian focal loss, radius from the box footprint.

**Decoder**

- Embedding 256, 8 heads, FFN 512, dropout 0.1.
- Layer 0 deformable BEV attention, 4 sampling points per query.
- Layer 1 SMCA over 4 FPN levels and 6 cameras, anisotropic $\sigma$ from the projected box size.
- Reference points and sizes updated after each layer, sizes detached after layer 0.

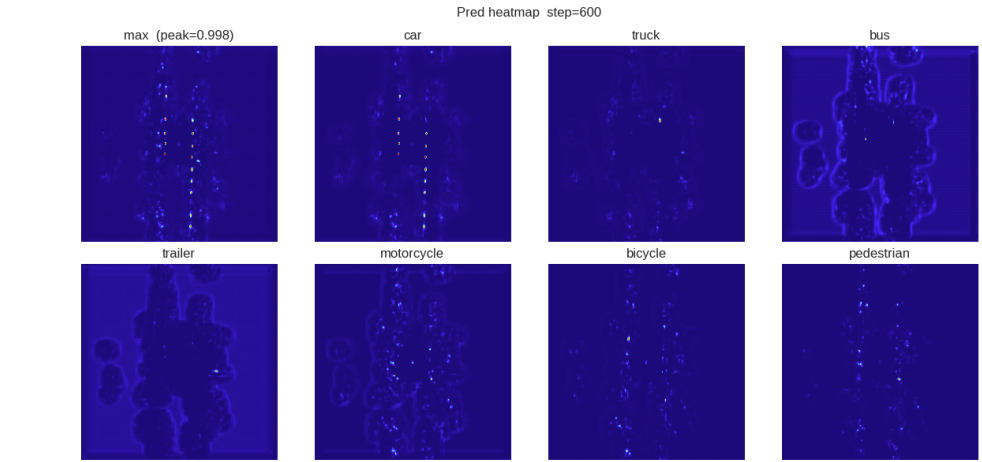
**Schedule**

- Stage 1: 20 epochs, camera path frozen.
- Stage 2: 6 epochs, LiDAR path frozen, lr $3.5\mathrm{e}{-4}$.

# Backup: questions for the lab

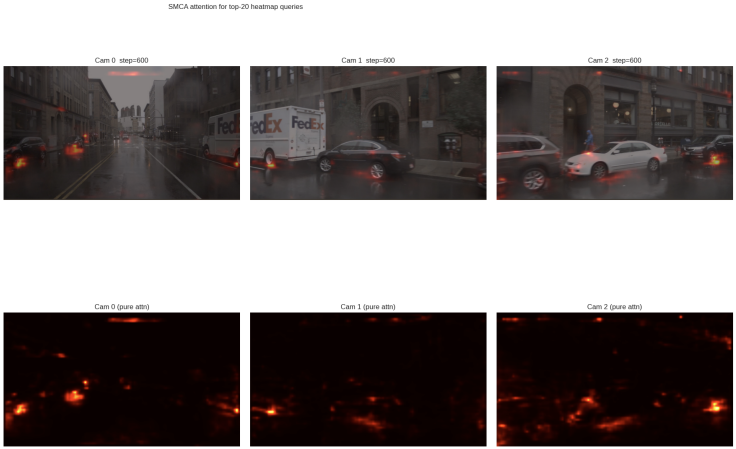
1. Which AR1 checkpoints and which parts of the Alpamayo training stack are available inside the lab, and is AlpaSim usable for closed-loop evaluation in February?
2. What compute resources are available?
3. Would I work in a team on the Alpamayo?
4. Is publication realistic?

# Backup: LiDAR BEV heatmap and predictions




Class-wise heatmap over the BEV grid, whose top-$K$ peaks become the agent queries.

# Backup: SMCA camera attention




Attention of the top 20 queries in decoder layer 1.

# References I


[1] L. Chen et al. "End-to-end autonomous driving: Challenges and frontiers". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46.12 (2024), pp. 10164–10183.

[2] M. Liang et al. "PnPNet: End-to-End Perception and Prediction with Tracking in the Loop". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2020, pp. 11553–11562.

[3] J. Gu et al. "ViP3D: End-to-End Visual Trajectory Prediction via 3D Agent Queries". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2023, pp. 5496–5506.

[4] N. Carion et al. "End-to-End Object Detection with Transformers". In: *European Conference on Computer Vision (ECCV)*. 2020, pp. 213–229.

[5] X. Zhu et al. "Deformable DETR: Deformable Transformers for End-to-End Object Detection". In: *International Conference on Learning Representations (ICLR)*. 2021.

[6] X. Bai et al. "TransFusion: Robust LiDAR-Camera Fusion for 3D Object Detection with Transformers". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2022.

[7] T. Yin et al. "Center-Based 3D Object Detection and Tracking". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2021.

# References II


[8] H. Caesar et al. "nuScenes: A Multimodal Dataset for Autonomous Driving". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2020.

[9] M. Halinkovic et al. *Li-ViP3D++: Query-Gated Deformable Camera–LiDAR Fusion for End-to-End Perception and Trajectory Prediction*. 2026. arXiv: 2601.20720 [cs.CV].

[10] M. Halinkovic et al. "Li-ViP3D: Enhancing End-to-End Perception and Prediction with Camera-LiDAR Fusion". In: *2025 IEEE International Automated Vehicle Validation Conference (IAVVC)*. 2025, pp. 1–6.

[11] NVIDIA AV Applied Research Team. *Alpamayo-R1: Bridging Reasoning and Action Prediction for Generalizable Autonomous Driving in the Long Tail*. 2025. arXiv: 2511.00088 [cs.CV].